

Vetores e Enumerações

Programação Imperativa

Prof. Pedro Rocha Horchulhack

Contato: p.horchulhack@pucpr.br

Agenda

1. Arrays e Vetores
2. Enumerações
3. *Pattern Matching*

Agenda

1. Arrays e Vetores

2. Enumerações

3. *Pattern Matching*

Arrays e Vetores

- É comum que trabalhem com dados em sequência;
- Em várias linguagens de programação trabalhamos com o conceito de arrays;
- Muitas situações exigem o conhecimento e organização de memória em **coleções**:
 - Usuários em um sistema;
 - Pixels de uma imagem;
 - Dados lidos em sequência;
 - Tabuleiros de jogos;
 - Notas da turma;
 - etc;
- Até o momento trabalhamos somente com a coleção String;
 - Coleção de caracteres (**char**);

Arrays e Vetores

- Na linguagem Rust temos três principais representações de sequências na memória:
 - Vetor estático (`[T; N]`): Armazenamos os dados em uma sequência de tamanho **conhecida (N)** de um único tipo (`T`);
 - Vetor dinâmico (`Vec<T>`): Dados armazenados em sequência, porém não é conhecido o tamanho em tempo de compilação. Tamanho variável, mas admite somente o tipo `T`;
 - *Slice* (`&[T]` e `&mut [T]`): Referência à uma parte de uma sequência de dados (vetor estático ou dinâmico).

Arrays e Vetores

Vetores estáticos

```
fn main() {  
    let fibonacci: [u32; 10] = [0, 1, 1, 2, 3, 5, 8, 13, 21, 34];  
    let linguagens = ["Python", "Rust", "C"];  
  
    for n in fibonacci {  
        println!("{n}");  
    }  
  
    // Acessar elemento específico com base no índice  
    println!("{}", fibonacci[1]);  
  
    // Tamanho do vetor  
    println!("{}", linguagens.len());  
}
```

Arrays e Vetores

Vetores estáticos – Inicializando com valor padrão

```
fn main() {  
    let fibonacci: [u32; 10] = [0; 10];  
  
    for n in fibonacci {  
        println!("{n}");  
    }  
}
```

Arrays e Vetores

Vetores estáticos – Vetor de vetores

```
fn main() {  
    let jogo: [[u8; 3]; 3] = [  
        [0, 0, 0],  
        [1, 1, 1],  
        [0, 0, 0]  
    ];  
  
    for linha in jogo {  
        for coluna in linha {  
            println!("{coluna}");  
        }  
    }  
}
```

Arrays e Vetores

Representação na memória

Representação no código

0	0	0
1	1	1
0	0	0

Representação na memória

0	0	0	1	1	1	0	0	0
---	---	---	---	---	---	---	---	---

Arrays e Vetores

Representação na memória

- Vetores estáticos têm esse nome devido ao **conhecimento** da quantidade de elementos em **tempo de compilação**;
- O cálculo é simples: tamanho do tipo * tamanho;
 - **u32**: $4 * 10 = 40\text{bytes}$
 - **u8**: $1 * 10 = 10\text{bytes}$
 - **i128**: $16 * 10 = 160\text{bytes}$
 - **f32**: $4 * 10 = 40\text{bytes}$

Arrays e Vetores

Representação na memória – Exercícios

Implemente o seguinte código:

- Crie um vetor estático de 5 elementos do tipo u8;
- Preencha o vetor com valores arbitrários;
- Itere por todos eles e mostre o valor de cada elemento;
- Junto ao valor, mostre o endereço na memória (referência) de cada valor;

Discuta com os colegas

Por que os endereços não atualizaram? O código está correto ou errado?

Resposta

Os endereços não atualizaram porque o acesso à um elemento do vetor considerará o endereço base (variável que armazena o vetor).

Arrays e Vetores

Vetores dinâmicos

- Em muitos casos **não** sabemos o tamanho da sequência de dados em tempo de compilação;
- Através de vetores dinâmicos conseguimos aumentar ou diminuir o vetor de acordo com a necessidade;
- Tipo `Vec<T>`, onde **T** é o tipo que queremos armazenar;
- Os dados são alocados na *heap*.

```
fn main() {  
    // Vetor dinâmico não inicializado  
    let v: Vec<u32>;  
  
    // Vetor dinâmico inicializado mas sem elementos  
    let mut w: Vec<u32> = Vec::new();  
  
    // Vetor dinâmico inicializado com elementos  
    let u: Vec<u32> = vec![1, 2, 3, 4];  
}
```

Arrays e Vetores

Vetores dinâmicos – Operações

```
fn main() {  
    let mut w: Vec<u32> = Vec::new();  
  
    // Inserir elemento ao final do vetor  
    w.push(3);  
    w.push(4);  
  
    // Insere na posição específica (operação lenta)  
    w.insert(1, 2);  
  
    // Remover elemento baseado no índice  
    w.remove(0);  
  
    // Acessar elemento baseado no índice  
    let i = w[0];  
  
    println!("Tamanho: {}", w.len());  
}
```

Arrays e Vetores

Slices

- É a região de um vetor estático/dinâmico;
- Pode assumir qualquer tamanho, desde que dentro dos intervalos de tamanho do vetor;
- É uma referência à uma região de memória contígua.

```
fn main() {  
    let w = vec![1, 2, 3, 4];  
    let sw: &[i32] = &w;  
}
```

Arrays e Vetores

Slices

- Podemos imprimir o conteúdo de uma *slice* ou vetor estático usando o formatador “:?”;
- Podemos indicar o intervalo de valores de um vetor. O resultado será um *slice*.

```
fn main() {  
    let w = vec![1, 2, 3, 4];  
  
    println!("{:?}", &w[1..3]); // [2, 3]  
}
```

Importante lembrar

O valor “1..3” dentro do *slice* é um Range.

Arrays e Vetores

Slices

Também podemos obter o *slice* de uma String!

```
fn main() {  
    let w: String = String::from("Ciberseguranca é o melhor curso");  
  
    println!("{:?}", &w[0..14]); // "Ciberseguranca"  
}
```

Arrays e Vetores I

Exercícios

1. Implemente uma função que receba a referência de uma variável do tipo `Vec<i32>`. Retorne a quantidade de elementos desse vetor. **Não é permitido utilizar a função `.len()`.**
2. Implemente uma função que receba um vetor estático de duas dimensões e mostre-o como uma tabela. Exemplo abaixo;

```
let v: [[u8; 5]; 5] = [[0; 5]; 5];
```

```
mostrar_matriz(&v);
```

```
/*
```

```
0 0 0 0 0
```

```
0 0 0 0 0
```

```
0 0 0 0 0
```

```
0 0 0 0 0
```

```
*/
```

Arrays e Vetores II

Exercícios

3. Implemente o jogo da velha, considerando um tabuleiro sendo um vetor estático de vetores estáticos. Sugestões:
- Crie funções (uma para cada situação) para validar se o jogador venceu, perdeu ou se deu velha;
 - Essas funções devem receber o tabuleiro como referência;
 - Utilize a função criada no Exercício 2.

Agenda

1. *Arrays e Vetores*

2. Enumerações

3. *Pattern Matching*

Enumerações

- Valores que podem, de maneira nominal, representar um tipo;
- Declaramos enumerações com a palavra chave `enum`;
- Utilizado quando queremos representar um conjunto de itens únicos, nomeados e constantes.

```
enum Veiculo {  
    Carro,  
    Onibus  
}  
  
fn main() {  
    let mut v: Veiculo = Veiculo::Carro;  
    v = Veiculo::Onibus;  
}
```

Enumerações

- Existem três tipos de enumerações:
 - Unitárias;
 - Estruturas;
 - Tuplas.

Enumerações

Enumerações unitárias

- Representadas por números inteiros (isize);
- Quando não possui valor associado, iniciam a contagem em 0;
- Podemos associar valores iniciais.

```
enum Veiculo {  
    Carro, // 0  
    Onibus // 1  
}  
  
fn main() {  
    let mut v: Veiculo = Veiculo::Carro;  
    v = Veiculo::Onibus;  
}
```

```
enum Veiculo {  
    Carro = 32,  
    Onibus = 10  
}  
  
fn main() {  
    let mut v: Veiculo = Veiculo::Carro;  
    v = Veiculo::Onibus;  
}
```

Enumerações

Enumerações tipo tupla

- Podemos associar valores baseados na sua **posição** em uma tupla.

```
enum Veiculo {  
    Carro (String, u8),  
    Onibus (String, u8)  
}  
  
fn main() {  
    // Placa e capacidade do tanque de combustível (litros)  
    let mut v: Veiculo = Veiculo::Carro("MWB7H21".to_string(), 35);  
    v = Veiculo::Onibus("ABD3F99".to_string(), 100);  
}
```

Enumerações

Enumerações tipo estrutura

- Associamos valores baseados em **campos nomeados**.

```
enum Veiculo {  
    Carro {placa: String, capacidade_tanque: u8},  
    Onibus {placa: String, capacidade_tanque: u8}  
}  
  
fn main() {  
    // Placa e capacidade do tanque de combustível (litros)  
    let mut v: Veiculo = Veiculo::Carro{placa: "MWB7H21".to_string(), capacidade_tanque:  
        ↪ 35};  
    v = Veiculo::Onibus{placa: "ABD3F99".to_string(), capacidade_tanque: 100};  
}
```

Enumerações

Enumerações tipo estrutura

- Enumerações **também** são considerados tipos;
- Podemos utilizar dentro de vetores!

```
enum Animal {  
    Cachorro, Gato,  
    Pato, Cavalo  
}  
  
fn main() {  
    let mut animais: Vec<Animal> = Vec::new();  
    animais.push(Animal::Gato);  
    animais.push(Animal::Gato);  
    animais.push(Animal::Cachorro);  
    animais.push(Animal::Pato);  
}
```

Agenda

1. Arrays e Vetores

2. Enumerações

3. *Pattern Matching*

Pattern Matching

- Na linguagem Rust temos a peculiaridade de enumerações não serem comparáveis (por padrão) através do operador ==;
- Isso acontece porque temos variantes de enumerações, portanto o compilador não saberá dizer qual a variante está sendo solicitada em tempo de execução.

```
enum Animal {  
    Cachorro,  
    Gato(bool),  
    Pato{raca: String}  
}
```

Pattern Matching

Discuta com os colegas

Como completamos o código dentro do laço de repetição “for” para obtermos os animais e os valores dos seus campos?

```
enum Animal {  
    Cachorro,  
    Gato(bool),  
    Pato{cor_pena: String}  
}  
  
fn main() {  
    let mut animais: Vec<Animal> = vec![];  
    animais.push(Animal::Gato(true)); // Dormindo  
    animais.push(Animal::Pato{cor_pena: "Branco".to_string()});  
    animais.push(Animal::Cachorro);  
  
    for animal in animais {  
        // ?  
    }  
}
```

Pattern Matching

Contra-exemplo

```
enum Animal {
    Cachorro,
    Gato(bool),
    Pato{cor_pena: String}
}

fn main() {
    let mut animais: Vec<Animal> = vec![];
    animais.push(Animal::Gato(true)); // Dormindo
    animais.push(Animal::Pato{cor_pena: "Branco".to_string()});
    animais.push(Animal::Cachorro);

    for animal in animais {
        if animal == Animal::Cachorro {
            println!("Isso é um cachorro");
        } else if animal == Animal::Gato {
            println!("O gato está dormindo? {}", ? );
        }
    }
}
```

Pattern Matching

Expressão match

```
fn formatar_animal(animal: Animal) -> String {  
    match animal {  
        Animal::Cachorro => format!("O animal é um cachorro!"),  
        Animal::Gato(dormindo) => {  
            let mut sim_nao = "sim";  
            if !dormindo { sim_nao = "não"}  
            format!("O animal é um gato e está dormindo? {}!", sim_nao)  
        },  
        Animal::Pato { cor_pena } => format!("O animal é um pato de cor {}", cor_pena)  
    }  
}
```

Pattern Matching

Expressão match

- **Padrões** consomem valores;
- **Expressões** produzem valores;

```
match expressao {  
  Animal::Cachorro => format!("O animal é um cachorro!"),  
}
```

Padrão (*pattern*) Expressão (retorno)

Pattern Matching

Expressão match

É possível utilizar com tipos primitivos

```
fn main() {  
    let nota = 5;  
    match nota {  
        1 => { println!("Horrível") },  
        2 => { println!("Ok...") },  
        3 => { println!("Razoável") },  
        4 => { println!("Bom") },  
        5 => { println!("Excelente") },  
        _ => { } // Aceita qualquer valor e não faz nada  
    }  
}
```

Pattern Matching

Expressão match

Verificando um intervalo

```
fn main() {  
    let nota = 5;  
    match nota {  
        1..=5 => {println!("Uma nota qualquer...")}  
        _ => {} // Aceita qualquer valor e não faz nada  
    }  
}
```

Pattern Matching

Expressão match

Verificando um intervalo de caracteres

```
fn main() {  
    let letra = 'a';  
    match letra {  
        'a' ..= 'z' | 'A' ..= 'Z' => {println!("Uma letra qualquer...")},  
        '0' ..= '9' => {println!("Um dígito qualquer...")},  
        _ => {} // Aceita qualquer valor e não faz nada  
    }  
}
```

Pattern Matching

Expressão match

Verificando um intervalo com *guards* (condições)

```
fn main() {  
    let idade = 18;  
    match idade {  
        i if i < 18 => { println!("Menor de idade") },  
        i if i >= 18 => { println!("Maior de idade") },  
        i if i >= 60 => { println!("Idoso") },  
        _ => {} // Aceita qualquer valor e não faz nada  
    }  
}
```

Pattern Matching

Exercícios

1. Crie uma enumeração chamada Voto que represente diferentes tipos de votos, como voto a favor, voto contra e voto em branco. Em seguida, crie uma função que aceite uma lista de votos representados como uma lista de variantes da enumeração. Use a correspondência de padrões (pattern matching) para contar e imprimir o número total de votos a favor, votos contra e votos em branco.
2. Crie uma enumeração chamada Carta que represente cartas de um baralho, incluindo naipes (paus, copas, espadas, ouros) e valores (Ás, 2 a 10, Valete, Dama, Rei). Em seguida, escreva uma função que aceite uma variante da enumeração Carta e retorne o valor da carta em pontos em um jogo de cartas simples. Use a correspondência de padrões para atribuir pontos com base na carta fornecida.